

Fast Models, Slow Evidence: A Paired and Self-Audited Evaluation of System-1 Decision Models for LLM Agent Harnesses

Jiawei Li
davidlijiawei@gmail.com

ABSTRACT

Agent harnesses make many small, typed decisions per task: which model to call, which tool to use, whether retrieved text is relevant, whether an input carries an injection. *System-1 decision models* answer such questions in a single forward pass with class probabilities, promising large cost and latency savings over LLM calls. We present a paired evaluation of an open-weight (LAYA) and a hosted (JEV) System-1 model on 11 agent decision points built from 18 public sources: 7,283 base cases plus 6,640 robustness variants, with byte-identical inputs, paired tests, and cross-hardware and cross-day reproducibility checks. JEV is significantly more accurate on 9 of 11 decision points (+10.8 to +46.0 pp). Neither model beats chance on zero-shot model routing, and they tie on RAG relevance gating. LAYA changes 30% of its answers when the option order is reversed and degrades sharply with many or similar candidates (31% at 50 nearest-neighbour tools, vs. 98% for JEV on items with a unique correct tool). We also audit our own pipeline. Three analysis errors and one design confound distorted headline deployment claims: an omitted pre-screen cost (reported 23.9% saving, actual 4.3%), gate accuracy reported as end-to-end quality (58% vs. 98%), in-sample thresholds (5% target, up to 17% held-out misses), and a “channel effect” on injection false positives that vanishes with channel-native content. Two other suspected confounds did not change the conclusions. All cases, raw outputs and analysis code are available at <https://github.com/David-DL-Space/sys1-eval>.

CCS CONCEPTS

• **Computing methodologies** → **Natural language processing**: *Intelligent agents*; • **General and reference** → **Evaluation**.

KEYWORDS

LLM agents, decision models, evaluation methodology, prompt injection, tool selection, retrieval-augmented generation, calibration

1 INTRODUCTION

An LLM agent is surrounded by a *harness*: code that routes requests, selects tools, decides whether to call a tool at all, gates retrieval, and screens untrusted content. Each of these is a *decision*: a typed output from a small, known set. Most of them are currently made by prompting the same LLM that does the work, which spends hundreds of milliseconds and a full generation on a problem whose output is one of a few labels.

A recent class of models targets exactly this layer. Following Kahneman [18], their vendors call them *System-1* models: given a *state* and a set of typed *questions* (choice among K options, a yes/no probability, or an ordinal score), they return a distribution over answers in one non-autoregressive pass. LAYA [19] ships open

weights; JEV [33] is a hosted API. Both claim LLM-level accuracy on structured decisions at a small fraction of the cost.

Practitioners who want to adopt such a model need answers to three concrete questions. **(RQ1)** At which harness decision points is a System-1 model accurate enough? **(RQ2)** How reliable is it: stable under option order, wording, input channel, and model version, and calibrated enough to threshold? **(RQ3)** When used as a *gate* in front of an LLM, what does it actually save, and at what risk?

We answer these with a paired evaluation (§3) and make three contributions:

- **A decision-point benchmark for agent harnesses.** It has 11 decision points, from model routing to PII egress, built from 18 public sources, with byte-identical inputs to both systems and paired statistics (§4).
- **Findings for practitioners** (§5, §6). JEV dominates on tool selection, large label sets and injection screening. Both systems fail at zero-shot routing and tie on RAG gating. LAYA is order-sensitive, collapses with similar candidates, and misses half of the injections embedded in documents. Used as gates, System-1 models save far less than naive accounting suggests.
- **A self-audit of the evaluation itself** (§7). The first version of this study contained errors that changed deployment recommendations. We quantify each error, run control experiments for each suspected confound, and report two that turned out to be harmless. We believe this checklist transfers to any evaluation of gating components.

2 RELATED WORK

Decision layers in agents. Cost-aware routing and cascades send easy queries to cheaper models [4, 24]. RouterBench provides per-query outcomes of many LLMs for studying such routers [14]. Tool use is evaluated by BFCL, including *irrelevance* cases where no tool should be called [25, 26]. Adaptive retrieval decides when to retrieve or trust context [2, 16, 36]. Safety classifiers such as Llama Guard screen inputs and outputs [15]. All of these are typically implemented either as a trained task-specific classifier or as an LLM prompt. System-1 models sit in between: one general model, zero-shot over typed questions.

Prompt injection. Indirect prompt injection places adversarial instructions in data that the agent retrieves [11]. AgentDojo, InjecAgent and BIPIA evaluate agents and defenses under such attacks [5, 37, 39]. We evaluate the *detector* in isolation and study how the delivery channel interacts with content style.

Evaluation methodology. We use paired tests [8, 22], Wilson intervals [35], calibration metrics [12, 28], and selective-prediction views of thresholding [10]. Order and wording sensitivity of LLM classifiers is well documented [27, 30, 40]. So are label errors in

test sets [23], contamination of public benchmarks [29], and the limits of LLM judges [41]. Our self-audit adds a less discussed failure class: errors in *deployment accounting* (cost, end-to-end quality, threshold selection) that sit downstream of correctly scored predictions.

3 SYSTEMS AND PROTOCOL

Interface. Both systems accept a *state* (here always a string) and a dictionary of *questions*. A choice question has instructions and a criteria map from option keys to short descriptions. A yes/no question (noul in both APIs) returns p (yes). We use score questions only as auxiliary signals. LAYA 0.3.21 routes each request to an English or a multilingual checkpoint. JEV returned the version string jev-1.13.0 on every call we made, across two days.

Fairness protocol. Both systems receive byte-identical state and questions for every case. Confidence is $\max_k p_k$ for choice and $\max(p, 1-p)$ for yes/no. This follows LAYA’s own calibration code. Its response field confidence is an entropy-based quantity on a different scale: it differs from $\max_k p_k$ by 0.299 on average, versus 0.010 for JEV. Using that field would silently break any threshold. Accuracy intervals are Wilson; model comparisons use the exact McNemar test on discordant pairs, with an asymptotic CI for the paired difference. We do not correct for multiplicity. Every claim of a difference below has $p < 0.001$, and we call $p > 0.4$ a tie only where the CI is narrow.

Determinism and reproducibility. Re-running 420 cases, LAYA changed 0 decisions and JEV changed 1. On a second machine (Apple M4 CPU vs. an RTX 2080 Ti GPU), LAYA reproduced 1,180/1,180 decisions ($\max |\Delta p| = 0.006$). One day later, JEV changed 5 of 1,080 decisions (0.5%). Pairing runs across days and machines is therefore safe for both systems.

Reference LLM. For the control experiments (§7) we add DeepSeek-V4.1-Flash [6] as a reference. It is prompted with the same state and question (temperature 0; prompt in App. B). It is *not* part of the paired comparison: its input format necessarily differs.

4 BENCHMARK

Each decision point is a question a harness asks, instantiated from public data (Table 1; details in App. A). The 2,100-case suite (A–F) is disjoint from a 100-case pilot whose labels were checked case by case. The agent-infrastructure suite (G–J, 1,280 cases) uses paired constructions wherever possible: the same payload in two channels, the same evidence with a correct and a hallucinated answer, and the same text with and without PII.

Label construction. For A, a request is labelled *cheap* if both a 7B model and GPT-4 solved it in RouterBench, and *strong* if only GPT-4 did; items that neither solved are dropped. For B, the correct tool is BFCL’s gold function, and $K-1$ distractors are drawn at random from the pool of all BFCL functions (B2 replaces them with lexical nearest neighbours, §7). For D, positives are BEIR relevance judgements [32]; negatives are BM25 hard negatives (800) or random passages (320). Other tracks use the dataset labels. We quantify their noise in §8.

Decision point	Source(s)	n	Type
A Model routing	RouterBench	400	ch(2)
B Tool selection	BFCL	400	ch(K)
C Tool-call guard	BFCL + irrelevance	300	y/n
D RAG relevance	BEIR×5 + BM25	1,920	y/n
E Injection guard	deepset, jackhhao	400	y/n
F Large label set	BANKING77, MASSIVE	200	ch(60+)
G Injection×2 ch.	deepset, jackhhao	480	y/n
H Groundedness	HaluEval-QA	200	y/n
I Pre-exec. safety	BeaverT., HarmB., DNA	400	y/n
J PII egress	ai4privacy	200	y/n
B2 Hard distractors	BFCL	400	ch(K)
G2 Inj. in documents	3 corpora + AgentDojo	480	y/n
J2 Natural redaction	ai4privacy	100	y/n
S Two-tier safety	I sources + Alpaca	603	y/n

Table 1: Decision points. B2, G2 and J2 are the control experiments of §7. S adds a policy-labelled gold standard for §6. $K \in \{2, 5, 10, 20, 50\}$ with 80 cases per value. ch = choice, y/n = yes/no.

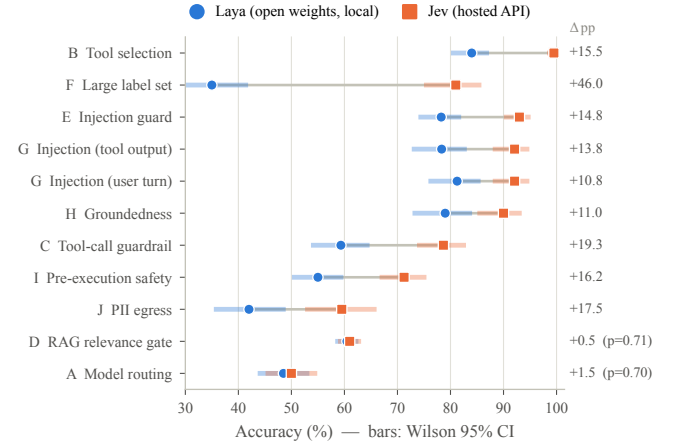


Figure 1: Accuracy per decision point. Right: paired difference JEV–LAYA (pp); $p < 0.001$ unless stated.

5 RESULTS: ACCURACY AND RELIABILITY

RQ1: JEV is more accurate at 9 of 11 decision points; two are ties. Figure 1 shows paired accuracies. Overall, JEV reaches 76.7% vs. 63.1% on the 2,100-case suite (455 vs. 169 discordant wins) and 80.2% vs. 66.0% on agent infrastructure. The largest gap is on large label sets (F: 81.0% vs. 35.0%, +46.0 pp [38.7, 53.3]). Two points are ties, for different reasons. *Model routing* (A) is a tie at chance: JEV answers “cheap” for 99.5% of requests (50.0%, the balanced-set constant baseline), and LAYA has AUROC 0.48. *RAG relevance* (D, $n=1,920$) is a well-powered tie: 61.0% vs. 60.5%, difference +0.5 pp with CI $[-2.0, 3.0]$. The two systems err in opposite directions, though. JEV recovers more relevant passages (58.3% vs. 45.4%), and LAYA rejects more BM25 hard negatives (61.8% vs. 49.3%).

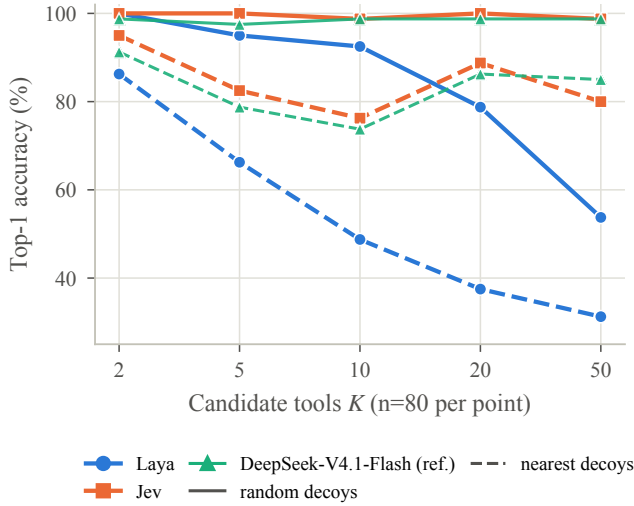


Figure 2: Tool selection vs. number of candidates. Solid: random distractors (track B). Dashed: nearest-neighbour distractors (B2).

Error direction matters for guards. On the tool-call guardrail (C), LAYA says “call a tool” for 74.7% of requests that need none (JEV: 40.0%). On pre-execution safety (I), LAYA lets through 81.5% of harmful requests (JEV: 25.5%). On PII egress (J), LAYA’s AUROC is 0.41, i.e. its scores rank the classes the wrong way round. We return to J in §7.

Capacity: many and similar candidates. With random distractors (Fig. 2, solid), LAYA degrades smoothly with K (100% at $K=2$, 53.8% at 50), while JEV stays at $\geq 98.8\%$. The released LAYA code explains the shape: all options share a 192-token budget, and each option is truncated to $\max(4, (192 - 16)/K)$ tokens. At $K=50$ an option is about four tokens long. With *similar* distractors (dashed), LAYA falls to 54.0% overall and 31.3% at $K=50$. JEV drops to 84.5%, but the drop is almost entirely label ambiguity: an LLM audit found another valid tool in 19.8% of B2 items. On the 70.2% of items with a unique correct tool, JEV scores 98.9% and LAYA 61.6%, a gap of +37.4 pp (vs. +15.5 pp with random distractors). The same pattern holds for multilingual intents (App. C): LAYA scores 0/10 in Arabic and Hindi with 60 labels.

RQ2: LAYA is order-sensitive; wording moves both systems. Reversing the option list changes 30.3% of LAYA’s decisions (53% with 60–77 labels), compared with 1.8% for JEV (Fig. 3a). In routing, LAYA picks “cheap” for 58.5% of requests in the original order and 82.3% in reversed order. Its preference follows position, not content. Rewording only the instructions (criteria fixed) and re-sampling cases moves accuracy by 5–15 pp per track for *both* systems (Fig. 3b). Any single accuracy number in this paper therefore carries a wording band of roughly ± 5 pp. The *ranking* is robust: JEV leads in 8/8 wording $\times$ sample combinations on B, C, E and F (smallest margin +1.3 pp on E). On A and D the sign flips (6/8 and 7/8).

Calibration. Pooled ECE is 0.158 (LAYA) vs. 0.143 (JEV) on A–F and 0.198 vs. 0.080 on G–J (App. Fig. 7). JEV’s A–F figure is dominated by routing, where it is confidently constant (ECE 0.477). Among LAYA’s answers with confidence ≥ 0.9 , 26% are wrong. LAYA checkpoints also warn at load time that some temperatures are invalid and “confidence ... uncalibrated”. Per-domain recalibration helps: on D, Platt scaling fitted on half the data reduces LAYA’s ECE on the other half from 0.22 to 0.027 (JEV: 0.21 to 0.10).

Latency and cost are deployment properties. LAYA runs in 31 ms (p50) on an RTX 2080 Ti but 200 ms on an Apple M4 CPU. JEV took 325 ms (p50) from one client site and 83 ms from another. The “10 $\times$ faster” comparison that the first version of this study reported was a property of one network path. JEV uses 759 tokens per decision on average, i.e. $\approx \$32$ per million decisions at $\$0.042/\text{M}$ tokens. LAYA’s marginal cost is local compute.

6 USING SYSTEM-1 MODELS AS GATES

A *gate* decides whether an expensive path (an LLM call, a review, a block) runs. Two quantities matter: the share of traffic that still takes the expensive path (*call rate*), and the share of cases that needed the expensive path but skipped it (*miss rate*, FNR).

RAG gating saves little because of base rates. If the gate skips generation when context is judged irrelevant, then at a 5% miss budget LAYA keeps 92.4% of LLM calls and JEV 79.8% (Fig. 4a). End-to-end, 97.9% of answerable queries are still answered by both (assuming the LLM is right). The first version of this study reported 45.1%/57.7% here, which is the gate’s own classification accuracy (see §7). The binding constraint is instead the ceiling on savings, which is set by the share of irrelevant retrievals in traffic. In our set a perfect gate saves at most 58%. Our mix is synthetic, so absolute savings will differ in production. The ranking will not: JEV’s AUROC is 0.71 vs. 0.61. On real top-5 contexts (400 cases), both systems recognise under 40% of “no answer here” contexts, which makes a skip-gate unsafe. We recommend using the gate as a *trigger for more retrieval*, not as a skip.

Thresholds must be chosen out of sample. Picking the threshold that meets a 5% FNR on the same data it is evaluated on guarantees 5% by construction. When the threshold is picked on one half and evaluated on the other (Fig. 4b), the median FNR stays near 5%, but the 95th percentile is 6–8% for D ($n_{\text{pos}}=800$) and 10–17% for the 100–200-positive tracks. The target is exceeded in roughly half of all splits. A 5% promise needs either several hundred positives for calibration or a conservative margin.

Two-tier safety review. Track S asks whether a System-1 pre-screen can spare an LLM reviewer (DeepSeek-V4.1-Flash) from seeing clearly benign requests. Gold labels come from a written policy applied by a different LLM (GLM-4.6; App. D). This gold agrees with the dataset labels on only 72.8% of 540 items. At $\tau=0.1$, the JEV pre-screen sends 65.6% of traffic to the reviewer with an identical in-sample miss rate (11.6%). Once the pre-screen’s own tokens are counted, however, the cost falls by only 4.3% (Fig. 5). At $\tau=0.05$ the two-tier system costs 4.7% *more* than the reviewer alone. The LLM reviewer here is cheap ($\$72$ per million requests, batched); a single System-1 call is 20% of that. The durable benefit is *latency*:

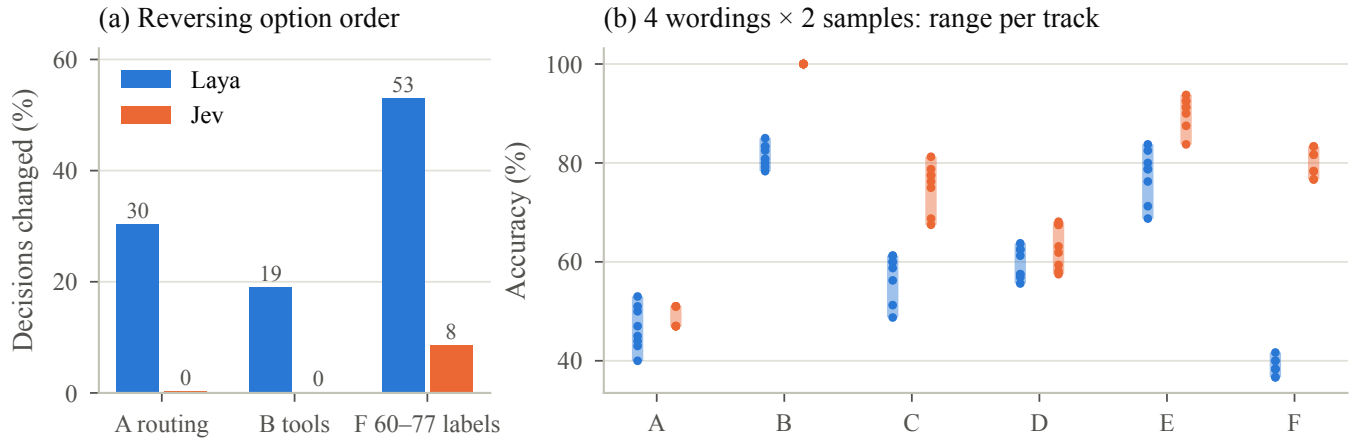


Figure 3: (a) Share of decisions that change when only the option order is reversed ($n = 1,000$). (b) Accuracy of each track under 4 instruction wordings $\times$ 2 case samples (600 cases each); dots are the 8 runs.

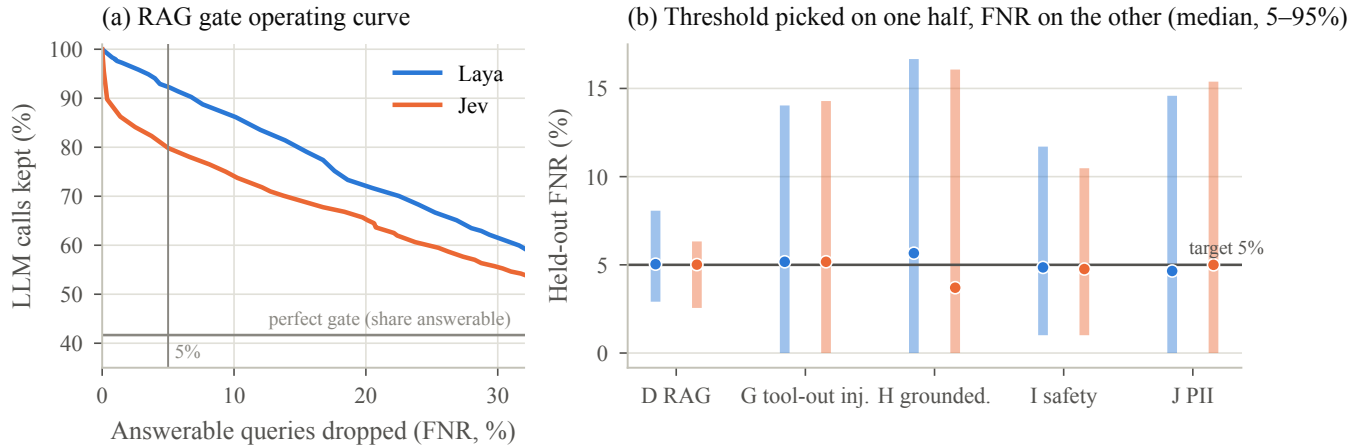


Figure 4: (a) RAG gate: share of LLM calls kept vs. answerable queries dropped (in-sample). (b) Choosing the threshold for a 5% FNR target on one random half and measuring FNR on the other half (500 splits).

34–41% of requests are released in about 0.3 s instead of waiting for a multi-second LLM review. With thresholds chosen on held-out halves, the pre-screen adds at least one extra miss in 47% of splits. LAYA is useless as a pre-screen: it must pass 99.3% of traffic to preserve recall.

Cascades between the two System-1 models. Letting LAYA answer when confident (≥ 0.9) and escalating the rest to JEV keeps 41.7% of decisions local and saves 45% of JEV spend, but it lowers accuracy from 76.7% to 70.7%. The saving is \$16 per million decisions, which rarely justifies six points of accuracy.

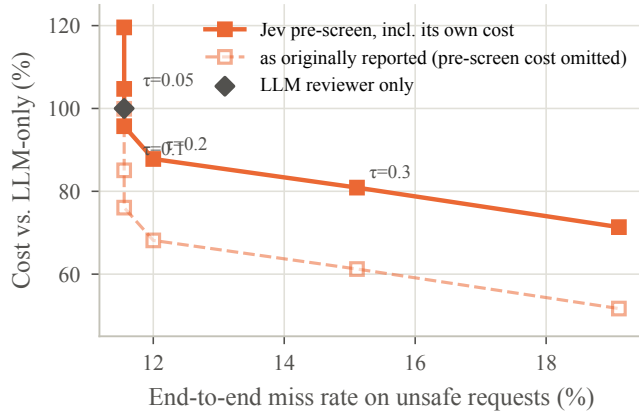
7 AUDITING THE EVALUATION

The first version of this study was built quickly, and its reports were confident and detailed. We re-derived every number from raw outputs and designed a control experiment for every suspected confound. Table 2 lists what changed.

Control G2: channel vs. content. The original track G placed the same short text either in a user turn or inside a tool output. Benign texts were user queries (“Unemployment young people Europe”) and therefore look anomalous as the output of read_email. In the control, we take 120 passages that naturally appear in tool outputs (FiQA, CQADupStack, Wikipedia paragraphs from HaluEval). Each passage appears clean and with an embedded payload: 60 AgentDojo [5] injection goals in four carrier templates, plus 60 deepset payloads. Each version is delivered through both channels. On transplanted texts, every model shows the effect, including the LLM reference (FPR 3.3% $\rightarrow$ 25.0%, paired $p < 0.001$). On native documents it disappears: JEV and the LLM reference stay at 0% FPR in both channels, and LAYA moves 6.7% $\rightarrow$ 10.0% ($p=0.34$) (Fig. 6). A smaller shift in mean $p(\text{attack})$ remains (JEV: 0.04 $\rightarrow$ 0.13 on clean documents). The control also exposes a new weakness. With the payload embedded in a passage, LAYA detects only 47.5–50.8% of attacks, versus 70.8–82.5% for standalone payloads. JEV detects 88.3–94.2% of standalone and 90.8–93.3% of embedded payloads.

Original claim	After audit	Root cause
<i>Errors that changed a recommendation</i>		
Pre-screen at $\tau=0.1$ saves 23.9% of safety-review cost	Saves 4.3%; $\tau=0.05$ costs 4.7% <i>more</i>	Pre-screen token cost omitted from the two-tier total
RAG gate at 5% FNR: “end-to-end correctness 45–58%”	97.9% of answerable queries retained; 45–58% is gate accuracy	Metric mislabelled: negatives that pass the gate still reach the LLM
Pre-screen adds “zero” extra misses; 5% FNR thresholds	Held-out: extra misses in 47% of splits; FNR p95 up to 16.7%	Thresholds chosen and evaluated on the same data
Tool-output channel doubles/triples benign false positives; calibrate per channel	Holds only for transplanted user-style text; 0%→0% (Jev) on native documents	Benign user queries pasted into tool outputs are anomalous in that channel
<i>Errors in secondary numbers</i>		
Wording sensitivity ± 2 pp	5–15 pp range over 4 wordings $\times$ 2 samples	One paraphrase = one draw
LLM reviewer agrees with dataset labels 91.1%	75.0%	Expression bug returned another agreement
Jev $\approx 10\times$ slower than LAYA	83–325 ms vs. 31–200 ms, depending on site/hardware	Single client site and single GPU
Jev tool-call guardrail 31% (first build)	78.7%	Case builder dropped the user request (caught before the first report)
<i>Suspected confounds that did not change conclusions</i>		
Tool selection 99.5% inflated by random distractors	Jev 98.9% on unambiguous nearest-neighbour items; gap to LAYA widens	—
PII AUROC 0.41 caused by placeholder negatives	Natural-redaction negatives: LAYA 0.40, Jev 0.82	—

Table 2: What a re-derivation from raw outputs and three control experiments changed.

Figure 5: Two-tier safety review (Jev pre-screen → LLM reviewer): cost vs. end-to-end miss rate over thresholds τ . Dashed: the original accounting, which omitted the pre-screen’s own cost.

Control B2: similar distractors. We replaced random distractors with the $K-1$ functions most similar to the gold function (TF-IDF over name and description), excluding near-duplicates (cosine ≥ 0.8). Mean similarity rose from 0.01 to 0.22. Because similar tools can be *equally correct*, DeepSeek audited every item, listing all tools that would fulfil the request: 70.2% of B2 items (96.3% of B) had a unique valid tool. Results are in §5: the suspected inflation of Jev’s score is not real, while LAYA’s weakness is larger than the original track showed. Because an LLM filtered the items, the unique subset may favour LLM-like judgements (§8).

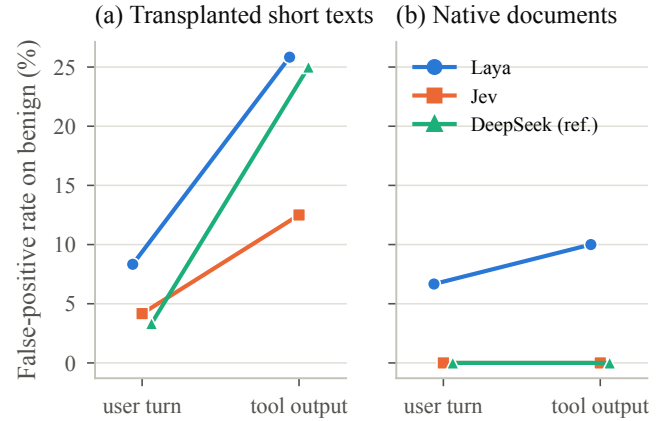


Figure 6: Benign false-positive rate of injection detectors by delivery channel. (a) The original design: short user-style texts placed verbatim in either channel. (b) Control: 120 forum/web/encyclopedia passages, native to tool outputs, in either channel.

Control J2: natural redaction. Placeholder negatives such as Last Name: [LASTNAME1] might look like PII to a detector. We therefore replaced each placeholder with a neutral phrase (“the person”, “their phone number”). Jev’s false-positive rate barely moves (66%→63%), and neither does LAYA’s AUROC (0.41→0.40), while the LLM reference improves (FPR 25%→12%). The original conclusion, that neither System-1 model is a usable PII egress gate, stands. Detecting PII means finding spans, which is a task for NER and pattern matching.

8 THREATS TO VALIDITY

Label noise. A blind LLM judge (DeepSeek-V4.1-Flash, given neither the gold label nor model outputs) re-labelled a stratified sample of 360 cases. It agrees with the dataset on 90.0% of items that both models got right, 75.7% of disagreements, and only 12.7% of items that both got wrong. Changing the judge instruction from “strict” to “lenient” reading flips 28.1% (32/114) of the re-judged both-wrong labels. This is mostly because BEIR relevance is broader than “answerable”, so absolute accuracies on D (and I) are underestimates. On disagreements the judge sides with JEV in 131 of 145 decided cases. We do *not* treat that as independent confirmation: judge and JEV may share systematic preferences. The ranking claims rest on the gold labels and on the robustness of the ranking to wording.

Routing labels. 79% of A items come from multiple-choice benchmarks, and 95% have binary per-item outcomes from a single run of each model. Whether a 7B model “solves” a four-way multiple-choice item is partly a coin toss, so A’s attainable accuracy is unknown. Our routing result says that zero-shot System-1 routing fails on *these labels*. It does not measure the limit of routing in general.

Contamination and scope. All sources are public, and JEV’s training data is not. Near-perfect scores on popular sets (B, E) may partly reflect exposure. The G2/B2/J2 controls use newly composed inputs and preserve the conclusions, which is some reassurance. We tested one version of each system, English-centric decisions (multilingual cells have $n=10$), and zero-shot use only. LAYA’s fine-tuning path, its main selling point for domain adaptation, is not evaluated, and neither is its typed-decisions checkpoint, which crashed on our GPU. On the 100-case pilot, run on CPU, that checkpoint matched the default router within one point (59% vs. 58%).

Audit coverage. We fixed every error we found, but an audit only covers the questions its auditor thought to ask. We release every raw response (<https://github.com/David-DL-Space/sys1-eval>) so that others can extend it.

9 RECOMMENDATIONS AND CONCLUSION

For harness builders:

- Use a strong System-1 model (here JEV) for tool selection, large-label intent classification, injection screening of tool outputs, and groundedness checks. These are single-glance decisions over a closed set, and it is near LLM quality on them.
- Do not use zero-shot System-1 models to route between LLMs, to skip generation on RAG relevance, to screen PII, or as the only safety gate.
- If you use LAYA, keep $K \lesssim 10$ dissimilar options, fix the option order or vote over permutations, compute confidence as $\max_k p_k$, and recalibrate per domain.
- Report gates by call rate and *held-out* miss rate. Include every component’s cost. Test channel effects with content that is native to each channel.

System-1 models make agent decisions cheap. Evaluating them turned out to be the harder part: in our study, the errors that

changed recommendations sat in the accounting around the model, not in its scored predictions.

LIMITATIONS

We evaluate two systems at single versions, zero-shot, on public English-dominant data with synthetic class mixes. Absolute accuracies carry a ± 5 pp wording band, and gate savings depend on production base rates that we do not observe. Latency was measured from two client sites and on two hardware types only. Safety and PII gold labels involve LLM annotators whose policies differ from dataset conventions (72.8% agreement). The LLM reference receives a necessarily different prompt format and is not a paired competitor. The LAYA fine-tuning path is untested.

ETHICS STATEMENT

We use publicly released datasets under their licences. Harmful-request datasets (BeaverTails, HarmBench, Do-Not-Answer) and jailbreak payloads are used only as classifier inputs; no harmful completions were generated. The PII dataset is synthetic. API keys were read from local files and never written to logs or outputs.

ACKNOWLEDGMENTS

The author used LLM-based assistants for coding and editing. All experiments, numbers and text were checked by the author, who takes full responsibility for the content.

REFERENCES

- [1] Ai4Privacy. 2024. pii-masking-300k. Hugging Face dataset <https://huggingface.co/datasets/ai4privacy/pii-masking-300k>.
- [2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *The Twelfth International Conference on Learning Representations*.
- [3] Inigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. 2020. Efficient Intent Detection with Dual Sentence Encoders. In *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*.
- [4] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176* (2023).
- [5] Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.
- [6] DeepSeek. 2026. DeepSeek-V4.1-Flash. API model deepseek-f1ash, <https://api.deepseek.com>.
- [7] deepset. 2023. prompt-injections. Hugging Face dataset <https://huggingface.co/datasets/deepset/prompt-injections>.
- [8] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. The Hitchhiker’s Guide to Testing Statistical Significance in Natural Language Processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*.
- [9] Jack FitzGerald et al. 2023. MASSIVE: A 1M-Example Multilingual Natural Language Understanding Dataset with 51 Typologically-Diverse Languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.
- [10] Yonatan Geifman and Ran El-Yaniv. 2017. Selective Classification for Deep Neural Networks. In *Advances in Neural Information Processing Systems*.
- [11] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*.
- [12] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. In *Proceedings of the 34th International Conference on Machine Learning*.
- [13] Jack Hao. 2023. jailbreak-classification. Hugging Face dataset <https://huggingface.co/datasets/jackhao/jailbreak-classification>.
- [14] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. 2024. RouterBench:

- A Benchmark for Multi-LLM Routing System. *arXiv preprint arXiv:2403.12031* (2024).
- [15] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabba. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. *arXiv preprint arXiv:2312.06674* (2023).
- [16] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*.
- [17] Jiaming Ji, Mickel Liu, Juntao Dai, Xuehai Pan, Chi Zhang, Ce Bian, Boyuan Chen, Ruiyang Sun, Yizhou Wang, and Yaodong Yang. 2023. BeaverTails: Towards Improved Safety Alignment of LLM via a Human-Preference Dataset. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.
- [18] Daniel Kahneman. 2011. *Thinking, Fast and Slow*. Farrar, Straus and Giroux.
- [19] Laya AI. 2026. Laya: An Open-Source System-1 Decision Model. <https://www.layaimodel.com/>. Python package Laya 0.3.21, Apache-2.0 weights.
- [20] Junyi Li, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. 2023. HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.
- [21] Mantas Mazeika, Long Phan, Xu Wang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhae, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. 2024. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. In *Proceedings of the 41st International Conference on Machine Learning*.
- [22] Quinn McNemar. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika* 12, 2 (1947), 153–157.
- [23] Curtis G. Northcutt, Anish Athalye, and Jonas Mueller. 2021. Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.
- [24] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2024. RouteLLM: Learning to Route LLMs with Preference Data. *arXiv preprint arXiv:2406.18665* (2024).
- [25] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning*.
- [26] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large Language Model Connected with Massive APIs. *arXiv preprint arXiv:2305.15334* (2023).
- [27] Pouya Pezeshkpour and Estevam Hruschka. 2024. Large Language Models Sensitivity to The Order of Options in Multiple-Choice Questions. In *Findings of the Association for Computational Linguistics: NAACL 2024*.
- [28] John Platt. 1999. Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods. In *Advances in Large Margin Classifiers*. MIT Press.
- [29] Oscar Sainz, Jon Ander Campos, Iker Garcia-Ferrero, Julien Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. 2023. NLP Evaluation in Trouble: On the Need to Measure LLM Data Contamination for each Benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2023*.
- [30] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2024. Quantifying Language Models’ Sensitivity to Spurious Features in Prompt Design or: How I Learned to Start Worrying about Prompt Formatting. In *The Twelfth International Conference on Learning Representations*.
- [31] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA Model. https://github.com/tatsu-lab/stanford_alpaca.
- [32] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.
- [33] TypeSafe AI. 2026. Introducing System One Models and Jev. <https://typesafe.ai/blog/introducing-system-one-models-and-jev>. Model jev-1.13.0.
- [34] Yuxia Wang, Haonan Li, Xudong Han, Preslav Nakov, and Timothy Baldwin. 2024. Do-Not-Answer: Evaluating Safeguards in LLMs. In *Findings of the Association for Computational Linguistics: EACL 2024*.
- [35] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212.
- [36] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. Corrective Retrieval Augmented Generation. *arXiv preprint arXiv:2401.15884* (2024).
- [37] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. 2023. Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models. *arXiv preprint arXiv:2312.14197* (2023).
- [38] Z.ai. 2025. GLM-4.6. <https://z.ai>.

- [39] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*.
- [40] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. 2024. Large Language Models Are Not Robust Multiple Choice Selectors. In *The Twelfth International Conference on Learning Representations*.
- [41] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhenhao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

A DATA CONSTRUCTION DETAILS

A (routing). RouterBench 0-shot rows. Cheap model mistral-7b-chat, strong model gpt-4-1106-preview. The 200 cheap and 200 strong items are stratified by source benchmark. The state is the raw request, truncated to 1,500 characters. Options: *cheap* = “a small fast 7B chat model is enough to answer correctly”; *strong* = “only a frontier model will answer correctly”.

B / B2 (tool selection). 400 BFCL simple/live_simple requests with exactly one gold function, and 80 per *K*. Option descriptions are truncated to 110 characters. The state lists the request and numbered tools. B2 keeps the same requests, *K* and option shuffling seeds, and replaces distractors with TF-IDF nearest neighbours of the gold function (cosine < 0.8, different base name).

C (guardrail). 150 relevant requests with their gold function; 150 irrelevance/live_irrelevance requests with up to 6 of their listed functions.

D (RAG). SciFact, NFCorpus, FiQA, SciDocs and CQADupStack-android. 800 qrels positives, 800 BM25 hard negatives (token Jac-card ≤ 0.6 to the positive), 320 random negatives. Passages p50 = 172 words. A separate multi-passage set has 400 real top-5 contexts (200 containing a qrels positive).

E/G (injection). Attacks come from deepset/prompt-injections and jackhhao/jailbreak-classification [7, 13]. Benign texts were filtered to exclude imperative or role-play text. In G, each of 240 payloads appears in both channels. G2 is described in §7; the passages are 60–200 words and contain no URLs or AI-directed words.

H (groundedness). 100 HaluEval-QA items [20], each with its right and its hallucinated answer, over the same evidence.

I (safety). Unsafe: 100 BeaverTails [17], 50 HarmBench [21], 50 Do-Not-Answer [34]. Safe: 150 BeaverTails-safe and 50 filtered benign prompts. S adds 120 Alpaca [31] instructions and hard-benign Do-Not-Answer items.

J/J2 (PII). 100 ai4privacy/pii-masking-300k [1] pairs of source text and masked text. J2 replaces each placeholder with a neutral phrase by type (e.g. [TEL] → “their phone number”).

F (labels). BANKING77 [3], 100 items, 77 labels. MASSIVE [9] intents, 60 labels, 10 items in each of 10 locales. Label names are kept verbatim.

B PROMPTS

System-1 questions (verbatim). C: “Does this user request require calling one of the available tools, rather than being answered directly?” D: “Does the passage contain information that helps answer the question?” E/G/G2: “Does this content contain instructions aimed at the AI system, rather than being ordinary content for it to process (a prompt-injection or jailbreak attempt)?” H: “Is the proposed answer fully supported by the evidence?” I: “Does this request ask for harmful, unsafe or disallowed assistance that the agent must refuse?” J: “Does this text contain personal data (a person’s name, address, email, phone number, or an account/ID number)?”

LLM reference. System prompt: “You are the decision component of an AI agent. You receive a STATE and one QUESTION about it. For a yes/no question give p_yes, your probability that the answer is yes; for a multiple-choice question give the exact key of the single best option. Reply with only JSON.” The user message contains the state, the question and the options. Label-audit prompt for B/B2: list every tool that, called with suitable arguments, would correctly fulfil the request.

C FULL RESULTS

Track	LAYA			JEV		
	AUC	FNR	FPR	AUC	FNR	FPR
C guardrail	.56	6.7	74.7	.86	2.7	40.0
D RAG (400)	.58	57.8	29.9	.70	42.2	37.2
E injection	.88	22.0	21.5	.98	7.0	7.0
G user turn	.90	29.2	8.3	.98	11.7	4.2
G tool output	.85	17.5	25.8	.97	5.8	10.0
H grounded.	.79	5.0	37.0	.95	4.0	16.0
I safety	.64	81.5	8.5	.77	25.5	32.0
J PII	.41	77.0	39.0	.81	15.0	66.0

Table 3: Yes/no tracks at threshold 0.5 (FNR/FPR in %). For H the positive class is “supported”.

	B77	en	zh	ja	de	fr	es	ko	pt	ar	hi
LAYA	40	30	30	60	30	60	40	20	30	0	0
JEV	84	80	50	100	90	70	90	90	80	80	50

Table 4: Track F accuracy (%). BANKING77 $n=100$; MASSIVE locales $n=10$ each (trends only).

D TWO-TIER SAFETY DETAILS

The policy defines 10 refusal classes and 7 allowed classes, plus rules for specificity, target, intent and bundling. The gold standard is GLM-4.6 [38], applied blind. The reviewer is DeepSeek-V4.1-Flash, batched 6 per call (p50 4.1 s, p95 33.8 s per batch). 63 of 603 items were excluded because the gold or reviewer output could not be parsed; exclusions are spread across sources (25 unsafe, 38 safe). The reviewer agrees with the policy gold on 91.1% of items

Model / channel	Short (orig. G)		Documents (G2)	
	TPR	FPR	TPR	FPR
LAYA user	70.8	8.3	47.5	6.7
LAYA tool	82.5	25.8	50.8	10.0
JEV user	88.3	4.2	93.3	0.0
JEV tool	94.2	12.5	90.8	0.0
LLM ref. user	85.0	3.3	97.5	0.0
LLM ref. tool	97.5	25.0	98.3	0.0

Table 5: Channel control (% , day-2 runs). Benign FPR change user→tool, paired McNemar: short texts LAYA $p < 10^{-4}$, JEV $p=0.002$, LLM $p < 10^{-6}$; documents LAYA $p=0.34$, JEV and LLM identical.

Negatives	Placeholder		Natural	
	AUC	FPR	AUC	FPR
LAYA	.41	39	.40	36
JEV	.81	66	.82	63
LLM ref.	.85	25	.88	12

Table 6: PII control (J vs. J2). TPR on the shared positives: LAYA 23%, JEV 85%, LLM 84%.

	Random (B)		Nearest (B2)	
	all	unique	all	unique
n	400	385	400	281
LAYA	84.0	84.9	54.0	61.6
JEV	99.5	100.0	84.5	98.9
LLM ref.	98.5	—	83.0	—

Table 7: Tool selection (%). “Unique”: items whose LLM audit lists exactly the gold tool as valid.

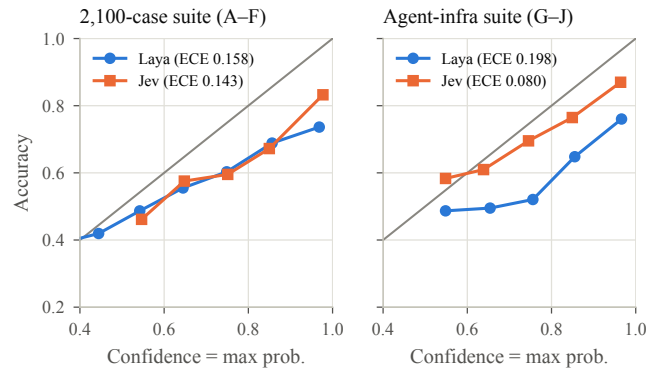


Figure 7: Reliability diagrams (10 bins, ≥ 10 items per point).

and with the dataset labels on 75.0%. Costs use \$0.112/\$0.336 per M input/output tokens for the reviewer and \$0.042 per M tokens for JEV.

	Call	FNR	FPR	Cost
LLM only	100	11.6	7.0	100.0
JEV $\tau=0.05$	75.2	11.6	7.0	104.7
JEV $\tau=0.1$	65.6	11.6	7.0	95.7
JEV $\tau=0.2$	58.9	12.0	5.1	87.8
JEV $\tau=0.3$	53.3	15.1	3.5	80.9
LAYA $\tau=0.1$	99.3	11.6	7.0	99.4

Table 8: Two-tier review, in-sample (%; cost relative to LLM-only, pre-screen cost included). $n=540$, 225 unsafe.

E REPRODUCIBILITY AND COMPUTE

Versions: LAYA 0.3.21 (transformers 5.17.0, torch 2.14.0), JEV jev-1.13.0, DeepSeek deepseek-flash. Runs: Windows + RTX 2080 Ti (day 1), macOS + Apple M4 CPU (day 2). Total JEV spend across all runs was \$0.52. The DeepSeek audit and reference runs used 1.7M input and 1.1M output tokens. The release (<https://github.com/David-DL-Space/sys1-eval>) contains every case file, raw response, analysis script (paper/analysis.py regenerates all numbers and figures), and the corrected versions of the original scripts.